

# A Practical Guide to the Smooth Periodic Gaussian HMM

SmoothPeriodicStatsModels.jl Extension

June 18, 2026

## Abstract

This guide presents a first implementation of a **Smooth Periodic Gaussian Hidden Markov Model** for continuous time series. The model extends a smooth periodic HMM framework by adding Gaussian/Normal emission distributions. Both the emission parameters and the transition probabilities are allowed to vary smoothly over a periodic index using a trigonometric basis. The guide is written to be practical and didactic: it explains the model, the main equations, the implementation structure, and how to run a minimal example in Julia.

## Contents

|                                                         |          |
|---------------------------------------------------------|----------|
| <b>1 Purpose of the Extension</b>                       | <b>2</b> |
| <b>2 Model Overview</b>                                 | <b>2</b> |
| <b>3 Trigonometric Basis</b>                            | <b>2</b> |
| <b>4 Gaussian Emission Model</b>                        | <b>3</b> |
| <b>5 Smooth Periodic Transition Probabilities</b>       | <b>4</b> |
| <b>6 Parameter Dimensions</b>                           | <b>4</b> |
| <b>7 Forward Algorithm</b>                              | <b>5</b> |
| <b>8 Backward Algorithm and Posterior Probabilities</b> | <b>5</b> |
| <b>9 Viterbi Decoding</b>                               | <b>6</b> |
| <b>10 Simulation</b>                                    | <b>6</b> |
| <b>11 Minimal Working Example</b>                       | <b>6</b> |
| <b>12 Recommended File Structure</b>                    | <b>7</b> |
| <b>13 Important Notes and Limitations</b>               | <b>8</b> |
| <b>14 Summary</b>                                       | <b>8</b> |

## 1 Purpose of the Extension

The goal of this extension is to support **continuous observations** in a smooth periodic Hidden Markov Model. In many applications, the observed variable is not binary, categorical, or bounded. Instead, it is a real-valued time series.

Examples of continuous time series include:

- energy inflow or electricity demand;
- prices and financial indicators;
- rainfall, temperature, or climate variables;
- reservoir levels or hydrological measurements;
- production, load, or engineering indicators.

A natural first continuous emission distribution is the Gaussian distribution. Therefore, this extension implements a model of the form:

$$Y_n \mid Z_n = k, \tau_n = t \sim \mathcal{N}(\mu_k(t), \sigma_k^2(t)), \quad (1)$$

where  $Z_n$  is the hidden state and  $\tau_n$  is the periodic index associated with observation  $n$ .

## 2 Model Overview

Let

$$y_1, y_2, \dots, y_N \in \mathbb{R} \quad (2)$$

be the observed time series. Each observation is associated with a periodic index

$$\tau_n \in \{1, 2, \dots, T\}. \quad (3)$$

For monthly data, for example,  $T = 12$  and  $\tau_n$  represents the month of the year. The model assumes a latent state sequence

$$Z_n \in \{1, 2, \dots, K\}, \quad (4)$$

where  $K$  is the number of hidden regimes.

The main idea is that the model parameters are not constant. Instead, they change smoothly with the periodic index  $t$ .

## 3 Trigonometric Basis

Smooth periodic functions are represented using a trigonometric basis. For a given period  $t$ , period length  $T$ , and number of harmonics  $d$ , define

$$x(t) = \left[ 1, \cos\left(\frac{2\pi t}{T}\right), \sin\left(\frac{2\pi t}{T}\right), \dots, \cos\left(\frac{2\pi dt}{T}\right), \sin\left(\frac{2\pi dt}{T}\right) \right]^\top. \quad (5)$$

The dimension of this vector is

$$\dim(x(t)) = 2d + 1. \quad (6)$$

In the implementation, this is produced by the internal function:

```
_sg_trig_basis(t, T, d)
```

For a sequence of periodic indices  $\tau_1, \dots, \tau_N$ , the design matrix is

$$X = \begin{bmatrix} x(\tau_1)^\top \\ x(\tau_2)^\top \\ \vdots \\ x(\tau_N)^\top \end{bmatrix} \in \mathbb{R}^{N \times (2d+1)}. \quad (7)$$

In the code, this is built by:

```
_sg_design_matrix(period_index, T, d)
```

## 4 Gaussian Emission Model

The emission model is Gaussian:

$$Y_n \mid Z_n = k, \tau_n = t \sim \mathcal{N}(\mu_k(t), \sigma_k^2(t)). \quad (8)$$

The smooth mean is parameterized as

$$\mu_k(t) = \theta_{\mu,k}^\top x(t), \quad (9)$$

where  $\theta_{\mu,k}$  is the vector of smooth mean coefficients for state  $k$ .

Written explicitly:

$$\mu_k(t) = \beta_{k,0} + \sum_{r=1}^d \left[ \beta_{k,r}^c \cos\left(\frac{2\pi r t}{T}\right) + \beta_{k,r}^s \sin\left(\frac{2\pi r t}{T}\right) \right]. \quad (10)$$

The standard deviation must be positive. Therefore, the model uses a log-scale parameterization:

$$\log \sigma_k(t) = \theta_{\sigma,k}^\top x(t), \quad (11)$$

or explicitly:

$$\log \sigma_k(t) = \gamma_{k,0} + \sum_{r=1}^d \left[ \gamma_{k,r}^c \cos\left(\frac{2\pi r t}{T}\right) + \gamma_{k,r}^s \sin\left(\frac{2\pi r t}{T}\right) \right]. \quad (12)$$

Thus,

$$\sigma_k(t) = \exp(\theta_{\sigma,k}^\top x(t)) > 0. \quad (13)$$

In the implementation, the emission parameters are computed by:

```
mu, sigma = smooth_gaussian_emission_parameters(model)
```

The emission log-likelihood matrix is computed by:

```
LL = smooth_gaussian_emission_loglikelihoods(model, y, period_index)
```

where

$$LL[n, k] = \log f_{\mathcal{N}}(y_n; \mu_k(\tau_n), \sigma_k^2(\tau_n)). \quad (14)$$

The Gaussian log-density is

$$\log f_{\mathcal{N}}(y; \mu, \sigma^2) = -\frac{1}{2} \log(2\pi) - \log \sigma - \frac{(y - \mu)^2}{2\sigma^2}. \quad (15)$$

## 5 Smooth Periodic Transition Probabilities

The hidden state sequence follows a time-inhomogeneous Markov chain:

$$P(Z_n = j \mid Z_{n-1} = i, \tau_n = t) = p_{ij}(t). \quad (16)$$

The transition probabilities are defined through a softmax function:

$$p_{ij}(t) = \frac{\exp(\eta_{ij}(t))}{\sum_{\ell=1}^K \exp(\eta_{i\ell}(t))}. \quad (17)$$

Each transition logit is also smooth and periodic:

$$\eta_{ij}(t) = \theta_{A,ij}^\top x(t). \quad (18)$$

Explicitly,

$$\eta_{ij}(t) = \alpha_{ij,0} + \sum_{r=1}^d \left[ \alpha_{ij,r}^c \cos\left(\frac{2\pi r t}{T}\right) + \alpha_{ij,r}^s \sin\left(\frac{2\pi r t}{T}\right) \right]. \quad (19)$$

For identifiability, the last state is used as the reference category:

$$\eta_{iK}(t) = 0. \quad (20)$$

Therefore, the implementation stores transition parameters only for  $j = 1, \dots, K-1$ .

The transition matrices are computed by:

```
A = smooth_gaussian_transition_matrices(model)
```

where

$$A[i, j, t] = p_{ij}(t). \quad (21)$$

The output has dimension

$$A \in \mathbb{R}^{K \times K \times T}. \quad (22)$$

## 6 Parameter Dimensions

The model is stored in a Julia structure named `SmoothGaussianHMM`:

```
mutable struct SmoothGaussianHMM
    K::Int
    T::Int
    d::Int
    pi0::Vector{Float64}
    theta_A::Array{Float64,3}
    theta_mu::Matrix{Float64}
    theta_logsigma::Matrix{Float64}
end
```

The dimensions are:

| Field                       | Meaning                             | Dimension                      |
|-----------------------------|-------------------------------------|--------------------------------|
| <code>pi0</code>            | initial state probabilities         | $K$                            |
| <code>theta_A</code>        | transition smooth coefficients      | $K \times (K-1) \times (2d+1)$ |
| <code>theta_mu</code>       | mean smooth coefficients            | $K \times (2d+1)$              |
| <code>theta_logsigma</code> | log-standard-deviation coefficients | $K \times (2d+1)$              |

## 7 Forward Algorithm

The forward variable is

$$\alpha_n(k) = P(y_1, \dots, y_n, Z_n = k). \quad (23)$$

Initialization:

$$\alpha_1(k) = \pi_k b_k(y_1, \tau_1), \quad (24)$$

where  $b_k(y_n, \tau_n)$  is the Gaussian emission density.

Recursion:

$$\alpha_n(j) = b_j(y_n, \tau_n) \sum_{i=1}^K \alpha_{n-1}(i) p_{ij}(\tau_n). \quad (25)$$

The likelihood is

$$L(\theta) = \sum_{k=1}^K \alpha_N(k). \quad (26)$$

The implementation uses the log-scale version to avoid numerical underflow:

$$\log \alpha_n(j) = \log b_j(y_n, \tau_n) + \log \sum_{i=1}^K \exp(\log \alpha_{n-1}(i) + \log p_{ij}(\tau_n)). \quad (27)$$

In Julia:

```
log_alpha, loglik, LL, A = smooth_gaussian_forward_log(model, y,
    period_index)
```

To return only the log-likelihood:

```
loglik = smooth_gaussian_loglikelihood(model, y, period_index)
```

## 8 Backward Algorithm and Posterior Probabilities

The backward variable is

$$\beta_n(i) = P(y_{n+1}, \dots, y_N \mid Z_n = i). \quad (28)$$

The terminal condition is

$$\beta_N(i) = 1. \quad (29)$$

The recursion is

$$\beta_n(i) = \sum_{j=1}^K p_{ij}(\tau_{n+1}) b_j(y_{n+1}, \tau_{n+1}) \beta_{n+1}(j). \quad (30)$$

The posterior probability of state  $k$  at time  $n$  is

$$\gamma_n(k) = P(Z_n = k \mid y_{1:N}) = \frac{\alpha_n(k) \beta_n(k)}{L(\theta)}. \quad (31)$$

The posterior probability of a transition from state  $i$  to state  $j$  is

$$\xi_n(i, j) = P(Z_{n-1} = i, Z_n = j \mid y_{1:N}). \quad (32)$$

In Julia:

```
gamma, xi, loglik = smooth_gaussian_posteriors(model, y, period_index)
```

## 9 Viterbi Decoding

The Viterbi algorithm returns the most likely hidden path:

$$\hat{z}_{1:N} = \arg \max_{z_{1:N}} P(z_{1:N} \mid y_{1:N}). \quad (33)$$

The dynamic programming recursion is

$$\delta_n(j) = \log b_j(y_n, \tau_n) + \max_i [\delta_{n-1}(i) + \log p_{ij}(\tau_n)]. \quad (34)$$

In Julia:

```
z_hat = smooth_gaussian_viterbi(model, y, period_index)
```

The vector `z_hat` contains one inferred state label for each observation.

## 10 Simulation

The simulation procedure is:

$$Z_1 \sim \text{Categorical}(\pi), \quad (35)$$

$$Y_1 \mid Z_1 = k, \tau_1 = t \sim \mathcal{N}(\mu_k(t), \sigma_k^2(t)), \quad (36)$$

$$Z_n \mid Z_{n-1} = i, \tau_n = t \sim \text{Categorical}(A[i, :, t]), \quad (37)$$

$$Y_n \mid Z_n = k, \tau_n = t \sim \mathcal{N}(\mu_k(t), \sigma_k^2(t)). \quad (38)$$

In Julia:

```
y_sim, z_sim = smooth_gaussian_rand(model, N)
```

or, with a custom periodic index:

```
y_sim, z_sim = smooth_gaussian_rand(model, N; period_index =  
    period_index)
```

## 11 Minimal Working Example

This example creates a toy continuous seasonal time series, initializes a Smooth Gaussian HMM, computes the log-likelihood, decodes states with Viterbi, and simulates new observations.

```
using SmoothPeriodicStatsModels
using Random
using Statistics

Random.seed!(123)

N = 300
T = 12
period_index = [mod(n - 1, T) + 1 for n in 1:N]

# Toy continuous seasonal data
```

```

y = [sin(2pi * period_index[n] / T) + 0.4 * randn() for n in 1:N]
y = (y .- mean(y)) ./ std(y)

model = initialize_smooth_gaussian_hmm(
    y,
    period_index;
    K = 3,
    T = 12,
    d = 1,
    p_stay = 0.90
)

A = smooth_gaussian_transition_matrices(model)
mu, sigma = smooth_gaussian_emission_parameters(model)
LL = smooth_gaussian_emission_loglikelihoods(model, y, period_index)
loglik = smooth_gaussian_loglikelihood(model, y, period_index)
gamma, xi, loglik2 = smooth_gaussian_posteriors(model, y, period_index)
z_hat = smooth_gaussian_viterbi(model, y, period_index)

y_sim, z_sim = smooth_gaussian_rand(model, N; period_index =
    period_index)

println(size(A))          # (K, K, T)
println(size(mu))         # (K, T)
println(size(sigma))      # (K, T)
println(size(LL))         # (N, K)
println(loglik)
println(counts(z_hat))

```

Expected dimensions:

```

size(A)      == (3, 3, 12)
size(mu)     == (3, 12)
size(sigma)  == (3, 12)
size(LL)     == (300, 3)

```

## 12 Recommended File Structure

A clean repository structure for this extension is:

```

SmoothPeriodicStatsModels.jl/
|
|-- src/
|   |-- SmoothPeriodicStatsModels.jl
|   |-- SmoothGaussianHMM.jl
|
|-- example/
|   |-- smooth_gaussian_hmm_mwe.jl
|
|-- test/
|   |-- runtests.jl
|
|-- docs/
|   |-- SmoothGaussianHMM_Guide.tex

```

The main package file should include the new model file:

```

include("SmoothGaussianHMM.jl")

```

It should also export the public functions:

```
export SmoothGaussianHMM
export initialize_smooth_gaussian_hmm
export smooth_gaussian_transition_matrices
export smooth_gaussian_emission_parameters
export smooth_gaussian_emission_loglikelihoods
export smooth_gaussian_forward_log
export smooth_gaussian_loglikelihood
export smooth_gaussian_backward_log
export smooth_gaussian_posteriors
export smooth_gaussian_viterbi
export smooth_gaussian_rand
```

## 13 Important Notes and Limitations

This first implementation focuses on the core model structure and inference utilities. It includes:

- smooth Gaussian emission parameters;
- smooth periodic transition matrices;
- Gaussian emission log-likelihoods;
- forward recursion in log scale;
- backward recursion in log scale;
- posterior probabilities;
- Viterbi decoding;
- simulation routines;
- quantile-based initialization.

A complete maximum likelihood estimation routine for all smooth parameters, especially a full EM/Baum–Welch M-step, can be developed as a future extension.

The current initialization is useful for testing, examples, and development, but should not be interpreted as a fully optimized fitting procedure.

## 14 Summary

The Smooth Periodic Gaussian HMM extends the smooth periodic HMM framework to continuous real-valued observations. The key modeling assumptions are:

$$Y_n \mid Z_n = k, \tau_n = t \sim \mathcal{N}(\mu_k(t), \sigma_k^2(t)), \quad (39)$$

$$\mu_k(t) = \theta_{\mu,k}^\top x(t), \quad (40)$$

$$\log \sigma_k(t) = \theta_{\sigma,k}^\top x(t), \quad (41)$$

$$p_{ij}(t) = \frac{\exp(\theta_{A,ij}^\top x(t))}{\sum_{\ell=1}^K \exp(\theta_{A,i\ell}^\top x(t))}. \quad (42)$$

This makes the model suitable for continuous time series with latent regimes and periodic structure.